

Лабораторна робота №4

Дисципліна: Основи програмної інженерії
Тема: Рефакторинг та Code Review
Студент: Коновалов Олександр, ПЗПІ-25-6
Викладач: Гребенюк В. О.

1. Тема та мета

Тема: Рефакторинг та Code Review

Мета роботи:

- Набути практичних навичок проведення Code Review чужого коду з виявленням структурних проблем та формуванням конструктивних рекомендацій.
- Навчитися ідентифікувати code smells (порушення DRY, dead code, поганий нейменування, недостатня валідація, відсутність документації) у реальних проектах.
- Виконати рефакторинг власного коду із забезпеченням регресійного тестування для підтвердження збереження коректної поведінки програми.

2. Результати Code Review одnogрупника

Репозиторій: `ruslanpetrechenko7/bse-lr3-petrechenko` **Файл:** `Index.js` — клас пошукового індексу (JavaScript, Jest)

№	Рядок коду	Проблема	Категорія	Рекомендація
1	L25: <code>q => result.add(q)</code>	Змінна <code>q</code> у <code>callback</code> не інформативна	Poor Naming	Перейменувати: <code>quote => result.add(quote)</code>
2	L3: <code>if (id <= 0)</code>	Конструктор не перевіряє тип параметра — <code>new Index("abc", 1)</code> не кине помилку, бо <code>"abc" <= 0</code> → <code>false</code>	Insufficient Validation	Додати: <code>typeof id !== 'number' !Number.isInteger(id)</code>
3	L13, L21	Патерн <code>!value value.trim() === ""</code> дублюється у <code>build()</code> та <code>search()</code>	Code Duplication (DRY)	Витягнути у допоміжну функцію <code>isBlank(str)</code>
4	L15-16	Дублювання цитат у індексі при повторенні слова в одній цитаті	Code Duplication (Data)	Перевірка <code>includes()</code> перед <code>push()</code>
5	L1-32	Відсутність JSDoc-документації на всіх публічних методах	Documentation Debt	Додати JSDoc з описом параметрів та повернених значень

3. Посилання на PR

Pull Request з результатами Code Review: <https://github.com/ruslanpetrechenko7/bse-lr3-petrechenko/pull/1>

4. Результати рефакторингу власного коду

Репозиторій: `oleksandrkonovalov1/bse-lr3-konovalov` (TypeScript, Vitest)

Операція 1: Усунення дублювання `login()` (DRY + Magic Number)

БУЛО:

```
// registered-user.ts (рядки 44-47) – ІДЕНТИЧНИЙ код
login(email: string, password: string): boolean {
  if (!email || !password) return false;
  return this.email === email && password.length >= 8;
}

// admin.ts (рядки 44-47) – ІДЕНТИЧНИЙ код
login(email: string, password: string): boolean {
  if (!email || !password) return false;
  return this.email === email && password.length >= 8;
}
```

СТАЛО:

```
// user.ts – базовий клас
const MIN_PASSWORD_LENGTH = 8;

export abstract class User {
  // ...
  login(email: string, password: string): boolean {
    if (!email || !password) return false;
    return this.email === email && password.length >= MIN_PASSWORD_LENGTH;
  }
}
```

ЧОМУ: Метод `login()` був скопійований побайтно у двох підкласах — порушення DRY. Магічне число `8` замінене на іменовану константу `MIN_PASSWORD_LENGTH`. Перенесення у базовий клас усуває дублювання та централізує логіку аутентифікації.

Операція 2: Видалення надлишкового методу `copyToClipboard()`

БУЛО:

```
// citation.ts – два методи з ідентичною логікою
getFormattedText(): string {
  return this.formattedText;
}

copyToClipboard(): string {
  return this.formattedText;
}
```

СТАЛО:

```
// citation.ts – тільки getFormattedText()
getFormattedText(): string {
  return this.formattedText;
}
// copyToClipboard() видалено
```

ЧОМУ: `copyToClipboard()` є dead code — повертає те саме значення, що й `getFormattedText()`. Два методи з різними іменами для однієї дії ускладнюють API та вводять в оману.

Операція 3: Інкапсуляція глобального лічильника ID

БУЛО:

```
// citation.ts — модульний рівень
let idCounter = 0;

export class Citation {
  constructor(metadata: PublicationMetadata, style: CitationStyle) {
    this.id = `cit-${++idCounter}`;
  }
}
```

СТАЛО:

```
export class Citation {
  private static nextId = 0;

  constructor(metadata: PublicationMetadata, style: CitationStyle) {
    this.id = `cit-${++Citation.nextId}`;
  }
}
```

ЧОМУ: Глобальна змінна `idCounter` на рівні модуля — це code smell "Excessive Complexity / Poor Encapsulation". Статичне поле класу чітко виражає належність лічильника до класу `Citation`, покращує інкапсуляцію та тестовність.

5. Звіт регресійного тестування

Після кожної операції рефакторингу всі 63 тести залишились зеленими:

- Операція 1: 63/63 passed
- Операція 2: 63/63 passed
- Операція 3: 63/63 passed
- Фінальне покриття: 96.02%

Фінальний результат тестування:

```
✓ src/crossref-provider.test.ts (4 tests)
✓ src/open-library-provider.test.ts (4 tests)
✓ src/web-scraper-provider.test.ts (4 tests)
✓ src/citation-style.test.ts (9 tests)
✓ src/search-request.test.ts (15 tests)
✓ src/citation-generator.test.ts (6 tests)
✓ src/user.test.ts (17 tests)
✓ src/citation.test.ts (4 tests)

Test Files  8 passed (8)
Tests      63 passed (63)
```

6. Порівняння метрик «ДО / ПІСЛЯ»

Метрика	ДО	ПІСЛЯ	Зміна
ESLint warnings	0	0	—
ESLint errors	0	0	—
Max Cyclomatic Complexity	10	10	—
Avg CCN	1.5	1.5	—
Тести (pass/total)	63/63	63/63	—
Coverage	96.02%	96.02%	—
Дубльовані методи login()	2	0	-2
Надлишкові методи (copyToClipboard)	1	0	-1
Глобальний мутабельний стан (idCounter)	1	0	-1

Примітка: ESLint і Cyclomatic Complexity не змінились, оскільки рефакторинг стосувався структурних проблем (дублювання коду, dead code, інкапсуляція), а не складності алгоритмів.

7. Підсумкова рефлексія

Code review навчив виявляти системні проблеми коду — не просто синтаксичні помилки чи баги, а структурні недоліки, які впливають на підтримуваність та розширюваність проекту. Порушення DRY у методі `login()` було невидимим для лінтерів (ESLint показував 0 попереджень), але відразу впадало в очі під час ручного огляду коду — це підтверджує незамінність людської перевірки. Рефакторинг із регресійним тестуванням виявився безпечним процесом: усі 63 тести слугували надійною страховочною сіткою, що гарантувала збереження поведінки програми після кожної зміни. Навіть зовні «чистий» код (0 ESLint warnings, 96% покриття) може мати структурні проблеми — dead code, порушення інкапсуляції, дублювання логіки — які знаходить лише уважний Code Review.